Obliczenia naukowe - Lista nr 1

Sprawozdanie z laboratorium

Jędrzej Stefanowski, 283948

Zadanie 1: Rozpoznanie arytmetyki

W Julii zaimplementowałem iteracyjnie wyznaczanie parametrów **macheps**, **eta** i **MAX** dla typów **Float16**, **Float32** i **Float64**. Wyniki zweryfikowałem za pomocą funkcji wbudowanych **eps**, **nextfloat** i **floatmax**. Porównałem je także ze stałymi z nagłówka **float.h** z **C**.

Parametr	Typ	Wynik w Julii	Dokładna postać
macheps	Float16	0.000977	2^{-10}
macheps	Float32	1.1920929e-7	2^{-23}
macheps	Float64	2.220446049250313e-16	2^{-52}
eta	Float16	6.0e-8	2^{-24}
eta	Float32	1.0e-45	2^{-149}
eta	Float64	5.0e-324	2^{-1074}
MINnor	Float32	1.1754944e-38	2^{-126}
MINnor	Float64	2.2250738585072014e-308	2^{-1022}
MAX	Float16	6.55e4	65 504
MAX	Float32	3.4028235e38	$(2 - 2^{-23}) \cdot 2^{127}$
MAX	Float64	1.7976931348623157e308	$(2 - 2^{-52}) \cdot 2^{1023}$

Wszystkie obliczone wartości były zgodne z wynikami odpowiednich funkcji wbudowanych Julii.

Porównanie z językiem **C**

Stała C	Wartość w C	Odpowiednik w Julii
FLT_EPSILON	1.1920929e-7	eps(Float32)
DBL_EPSILON	2.220446049250313e-16	eps(Float64)
FLT_MIN	1.1754944e-38	floatmin(Float32)
DBL_MIN	2.2250738585072014e-308	floatmin(Float64)
FLT_MAX	3.4028235e38	floatmax(Float32)
DBL_MAX	1.7976931348623157e308	floatmax(Float64)

*Wartości parametrów **Float32** i **Float64** są zgodne z odpowiednimi stałymi z pliku **float.h** języka **C**. Stałe **FLT_MIN** i **DBL_MIN** odpowiadają najmniejszym dodatnim liczbom znormalizowanym (**MINnor**), a nie najmniejszym liczbom podnormalnym (**eta**).*

Wnioski:

- **macheps** odpowiada odstępowi między liczbą **1** a jej **następnikiem**. Dla zaokrąglania do najbliższej zachodzi zależność **macheps** $= \mathbf{2}\varepsilon$, gdzie ε to precyzja arytmetyki z wykładu.
- Wyznaczona wartość **eta** jest najmniejszą dodatnią liczbą podnormalną (**MINsub**), natomiast **floatmin(T)** zwraca najmniejszą dodatnią liczbę znormalizowaną (**MINnor**).
- Zwiększenie precyzji typu zmniejsza odstęp między reprezentowalnymi liczbami w otoczeniu jedynki. **Float64** ma również znacznie szerszy zakres wartości niż **Float32**.

Zadanie 2: Rozmieszczenie liczb Float64

Wyznaczyłem odstępy między kolejnymi liczbami maszynowymi w przedziałach [0,5; 1], [1; 2] i [2; 4] za pomocą funkcji **nextfloat()** i **prevfloat()**. Następnie sprawdziłem reprezentację bitową kolejnych liczb przy użyciu **bitstring()**.

Przedział	Odstęp (wynik Julii)	Postać dokładna
[0,5; 1]	1.1102230246251565e-16	2^{-53}
[1; 2]	2.220446049250313e-16	2^{-52}
[2; 4]	4.440892098500626e-16	2^{-51}

W każdym przedziale odstęp na początku (**nextfloat(a) - a**) był równy odstępowi przy jego prawym końcu (**b - prevfloat(b)**).

Analiza bitstring(). Dla liczb $x = 1 + k \cdot 2^{-52}$, gdzie $k = 0, 1, 2, 3$, bit znaku wynosił **0**. Zapisany wykładnik miał postać **01111111111** co odpowiada wartość $E = 1023$ i rzeczywistemu wykładnikowi $c = E - 1023 = 0$. Końcowe bity mantysy przyjmowały kolejno wartości **00, 01, 10, 11**, zgodnie z binarnym zapisem k.

Wnioski:

- W każdym badanym przedziale kolejne liczby **Float64** są rozmieszczone równomiernie. Po przejściu do przedziału o **dwukrotnie większych** wartościach odstęp między kolejnymi liczbami również się **podwaja**.
- Liczby maszynowe w badanych przedziałach mają odpowiednio postać $x = 0.5 + k \cdot 2^{-53}$, $x = 1 + k \cdot 2^{-52}$, oraz $x = 2 + k \cdot 2^{-51}$

Zadanie 3. Błędy zaokrągleń

W arytmetyce **Float64** przeszukałem kolejne liczby maszynowe z przedziału (1, 2), sprawdzając warunek $x \cdot \dfrac{1}{x} \neq 1$.

Parametr	Wartość
Najmniejsze znalezione x	1.000000057228997
Numer k	257736490
Obliczone $\dfrac{1}{x}$	0.9999999427710061
Wynik $x \cdot \dfrac{1}{x}$	0.9999999999999999
Błąd bezwzględny	1.1102230246251565e-16 $= 2^{-53}$

Wnioski:

- Najmniejszą liczbę spełniającą warunek znaleziono przez przegląd kolejnych liczb maszynowych od **nextfloat(1.0)**.
- Matematyczna tożsamość $x \cdot \frac{1}{x} = 1$ nie musi zachodzić dokładnie w arytmetyce zmiennoprzecinkowej wskutek zaokrągleń podczas dzielenia i mnożenia.
- Dla poprzedniej liczby maszynowej **1.0000000572289969** otrzymano wynik **1.0**. Ponieważ algorytm sprawdzał kolejno wszystkie liczby od **nextfloat(1.0)**, znalezione x jest najmniejszą liczbą spełniającą warunek.

Zadanie 4. Wpływ kolejności sumowania na dokładność iloczynu skalarnego

Obliczyłem iloczyn skalarny podanych wektorów w arytmetykach **Float32** i **Float64**, stosując cztery sposoby sumowania: **(a) w przód**, **(b) w tył**, **(c) od największych do najmniejszych** wartości bezwzględnych oraz **(d) w odwrotnej kolejności**. W metodach **(c)** i **(d)** składniki dodatnie i ujemne sumowano osobno.

Wartość referencyjna: $S = -1.00657107 \cdot 10^{-11}$.

Typ	Metoda	Wynik	Błąd bezwzględny	Błąd względny
Float32	(a)	-0.4999443	$4.9994 \cdot 10^{-1}$	$4.9668 \cdot 10^{10}$
Float32	(b)	-0.4543457	$4.5435 \cdot 10^{-1}$	$4.5138 \cdot 10^{10}$
Float32	(c)	-0.5	$5.0000 \cdot 10^{-1}$	$4.9674 \cdot 10^{10}$
Float32	(d)	-0.5	$5.0000 \cdot 10^{-1}$	$4.9674 \cdot 10^{10}$
Float64	(a)	$1.02519 \cdot 10^{-10}$	$1.12585 \cdot 10^{-10}$	11.185
Float64	(b)	$-1.56433 \cdot 10^{-10}$	$1.46367 \cdot 10^{-10}$	14.541
Float64	(c)	0.0	$1.00657 \cdot 10^{-11}$	1.0
Float64	(d)	0.0	$1.00657 \cdot 10^{-11}$	1.0

Wnioski:

- Kolejność sumowania wpływa na wynik obliczeń zmiennoprzecinkowych. Dla **Float32** najmniejszy błąd uzyskano metodą **(b)**, a dla **Float64** metodami **(c)** i **(d)**.
- Znaczna utrata dokładności wynika z niemal całkowitego znoszenia się dodatnich i ujemnych składników o sumach rzędu $2{,}76 \cdot 10^6$, których suma jest bliska zeru.
- **Float64 zapewnia mniejsze błędy bezwzględne niż Float32**, jednak nawet najlepsze wyniki **Float64** mają **błąd względny równy 100%**. Większa precyzja nie eliminuje problemu utraty cyfr znaczących.

Zadanie 5. Utrata cyfr znaczących

W arytmetyce **Float64** obliczyłem wartości matematycznie równoważnych funkcji

$$f(x) = \sqrt{1 + x^2} - 1 \qquad \text{oraz} \qquad g(x) = \frac{x^2}{\sqrt{1 + x^2} + 1}$$

dla $x = 8^{-n}$, gdzie $n = 1, \ldots, 15$. Dodatkowo zbadałem wykładniki x^2 za pomocą exponent() i porównałem je z odstępem 2^{-52} między kolejnymi liczbami w pobliżu 1.

Wybrane wyniki:

n	$f(x)$	$g(x)$
1	0.0077822185373186414	0.0077822185373187065
8	1.7763568394002505e-15	1.7763568394002489e-15
9	0.0	2.7755575615628914e-17
10	0.0	4.336808689942018e-19
15	0.0	4.0389678347315804e-28

Dla $n = 9$ otrzymałem $x = 2^{-27}$ oraz $x^2 = 2^{-54}$. Ponieważ odstęp między 1 a kolejną liczbą Float64 wynosi 2^{-52}, dodawane x^2 stanowi tylko $\frac{1}{4}$ tego odstępu. Wskutek zaokrąglenia $1 + 2^{-54}$ daje **1.0**, dlatego $f(x)$ zwraca **zero**.

Wnioski:

- Funkcja $f(x)$ traci dokładność wskutek odejmowania liczb bliskich sobie. **Od n = 9** zwraca błędne **zero**.
- Funkcja $g(x)$ unika tego odejmowania i zachowuje wiarygodne wyniki dla wszystkich badanych argumentów. Przekształcenie algebraiczne poprawia stabilność numeryczną obliczeń.

Zadanie 6. Przybliżanie pochodnej

Metoda. Obliczono przybliżenie pochodnej funkcji $f(x) = \sin(x) + \cos(3x)$ w punkcie $x_0 = 1$, stosując wzór

$$\frac{f(1+h) - f(1)}{h}$$

dla $h = 2^{-n}$, $n = 0, \ldots, 54$, w arytmetyce Float64.

Wartość pochodnej ze wzoru analitycznego:

$$f'(1) = \cos(1) - 3\sin(3) \approx 0.11694228168853815.$$

Wybrane wyniki:

n	Przybliżenie pochodnej	Błąd bezwzględny
0	2.0179892253	1.9010469436
20	0.1169461290	3.847323e-6
28	0.1169422865	4.802856e-9
29	0.1169422269	5.480179e-8
40	0.1168212891	1.209926e-4
50	0.0	0.1169422817
52	−0.5	0.6169422817
53	0.0	0.1169422817

Wnioski:

- Najmniejszy błąd uzyskano dla $n = 28$ ($h \approx 3.72529 \cdot 10^{-9}$), wynosił on około $4.80 \cdot 10^{-9}$. Dalsze zmniejszanie h nie poprawiało systematycznie dokładności z powodu błędów zaokrągleń i odejmowania bliskich wartości funkcji.
- Dla $n = 50$ przybliżenie wyniosło 0.0, mimo że $1 + h \neq 1$. Dla $n = 53$ i 54 dodawanie h do jedynki zostało zaokrąglone do 1.0, co również spowodowało zerowy wynik.
- Dla dużych wartości h dominuje błąd przybliżenia pochodnej, natomiast dla bardzo małych h istotniejsze stają się **błędy zaokrągleń**. Odejmowanie zbliżonych wartości funkcji oraz dzielenie ich różnicy przez mały krok mogą znacząco pogarszać dokładność obliczeń.